Lift: Multi-Agent Legacy Code Migration, Full Implementation Plan

A zero-to-end build plan with reasons and alternatives at every step. October 29, 2026.

How to read this plan

Part 1 (sections 1 to 7) explains the project: what it is, why it matters, what real companies did, how yours is different, and what it shows about you. Part 2 (phases 0 to 16) is the build plan. Every phase uses the same four blocks: **What we build**, **Why we do it**, **Other options**, **Done when**. If you disagree with a choice, the Other options block tells you what to swap in and what you lose.

The most important rule: build one thin vertical slice first (Python 2 to 3, from input repo to verified output), and only then widen to the service layer and the second recipe. A narrow thing that works end to end beats a wide thing that half works.

Working name: **Lift**. Scope: one pipeline, two migration recipes (Python 2 to 3, and Express to FastAPI), delivered as a backend service.

1. What the project is

One line: you give Lift an old codebase and a target, and a team of agents, coordinated by a deterministic orchestrator, converts it chunk by chunk and proves the new code behaves like the old code by running tests that were recorded on the old code first.

Input: a repo (zip or git URL) plus a recipe name, for example `py2-to-py3` or `express-to-fastapi`.

Output:

- the migrated repo (on a branch or as a zip)
- a report: how many chunks passed first try, passed after retries, and went to human review (with the reason for each)
- cost and time: LLM calls, tokens, seconds
- a trace: which agent did what, with which tool, and how long it took

The core idea (remember this one): tests are the judge, not the LLM. Before any code changes, we record what the old program actually does (inputs and outputs). The new code must reproduce those outputs. This is called characterization testing, or the golden master approach. The LLM only proposes code and test inputs. Real execution decides what is correct.

Example A: why syntax fixes are not enough (Python 2 to 3)

```
# Python 2
def average(nums):
    return sum(nums) / len(nums)

print average([1, 2, 4])   # prints 2
```

A quick fix adds brackets to `print`. The file now parses in Python 3, but `average([1, 2, 4])` prints `2.3333...` instead of `2`, because `/` changed from integer division to true division. A syntax-only tool would call this migrated. Lift's recorded test says the old output was `2`, so the new code fails, and the system must choose `//` to preserve behavior. It should also flag the line for human review, because the old behavior may have been a bug.

Example B: why framework migrations are not 1 to 1 (Express to FastAPI)

```
// Express
app.get('/users/:id', (req, res) => {
  const user = db.find(req.params.id);
  if (!user) return res.status(404).json({ error: 'not found' });
  res.json(user);
});
```

The obvious FastAPI version is `@app.get('/users/{id}')` with `raise HTTPException(404)`. But FastAPI returns `{'detail': ...}` for errors, not `{'error': ...}`, and it returns a 422 with its own shape when a path or query value fails validation. Any client that reads `error` now breaks. Recorded HTTP tests catch this. Because HTTP tests do not care what language the server is written in, the same test suite runs against the old and the new app.

2. The friction it removes

The problem in plain words: old code keeps running in production long after its language, framework, or library stops getting support. Moving it is slow, boring, and risky, because one small behavior change can break users. Nobody wants to do it by hand, and the people who understand the old stack are leaving.

Real examples:

- **Python 2 reached end of life on 1 January 2020.** It stopped getting security fixes, but a lot of internal tools and services kept running on it for years afterwards. Every one of those needs a migration.
- **Amazon documented moving millions of lines of Java from Java 8 to Java 17 with LLM help.** Version upgrades are a regular chore for any company with many services.
- **A Dutch financial institution has about 2.5 million lines of PL/SQL** with inconsistent documentation and no automated tests, and researchers built a multi-agent translation system for it.
- **A shopping-mall ERP written in Visual Basic 6 and running for over two decades** was the subject of a 2026 case study that migrated 12 features to C# .NET 10 using an AI coding agent.
- **AWS CardDemo, about 50,000 lines of COBOL,** is used by open-source projects as a public migration benchmark.

Who feels this pain:

Who	Their version of the pain
Banks, insurers, large enterprises	COBOL, PL/SQL, VB6 systems, few engineers who know them
Platform and infra teams	Hundreds of services stuck on old runtimes and libraries
Startups	An old Express or Flask monolith that nobody wants to touch
Open-source maintainers	Old projects that need a new framework version but have no time
Students and freelancers	Inheriting an old project with no documentation

Honest framing: this is developer and company friction, not everyday-life friction. That is fine for backend and platform roles, but do not pitch it as something your parents would use.

3. What real companies and open source have already done

The numbers below come from blog posts, papers, and secondhand summaries that I read while researching. Check the original source before you quote any figure in an interview.

Who	What they did	What you can learn from it
Google	Described using LLMs for internal migrations: find code to change, write the new version, run tests, iterate until tests pass, engineers review. One summary says it saved over half the time on a very large migration.	The loop (propose, test, retry) and human review at the end
Amazon	Amazon Q Code Transformation upgrades Java versions. It makes a plan, builds, and runs existing unit tests to fix errors step by step. An outside analysis says deterministic OpenRewrite recipes likely did most of the changes, with the LLM handling the leftovers.	Rules first, LLM second. Cheaper and more reliable
Airbnb	Used LLMs to speed up a large test migration.	Test files are a good first migration target

Who	What they did	What you can learn from it
Microsoft (COBOL Agentic Migration Factory)	Three worker agents (COBOL expert, Java expert, test expert) built on AutoGen and later Semantic Kernel. They found that deterministic controls, especially tests, were crucial.	Role split, and tests as the control
Dutch financial institution (LegacyTranslate paper)	Three agents: initial translation, API grounding, and a third for framework-level constraints. Built for about 2.5M lines of PL/SQL.	Translation that compiles is not enough, it must fit the existing APIs
VB6 ERP case study (2026)	One coding agent migrated 12 features of different complexity to C# .NET 10.	Complexity of the code limits success, so report results per difficulty
Aviator (Java to TypeScript)	Three agents: file reader, planner, migrator.	The same planner/migrator split we will use
Products	IBM watsonx Code Assistant for Z, GitHub Copilot Modernization, Gemini Code Assist.	The problem is commercial, which proves demand

Open-source repos worth reading (read the README and benchmark, do not copy code):

- **gkhurana21/codeshift:** Python 2 to 3 agent verified against a 41-case behavioral oracle, 95.1% pass rate. Best reference for how to report results.
- **asharma157/tech-stack-upgrade:** Spring Boot 2 to 3 migration that ends in a GitHub PR. Migrations are added as recipe folders, which is the pluggable idea we use.
- **LegacyLens (Asaif Ali):** multi-agent platform with program analysis, risk scoring, FastAPI and Streamlit.
- **VAPU and Multi-agent-pipeline (research code):** multi-agent updaters evaluated on real Python projects.

- **Bernstein:** a deterministic scheduler running one agent per module with equivalence tests.
- **AZClaw:** small local-model agents migrating CardDemo COBOL to Python.
- **codegen-sh/codegen-examples:** non-LLM codemod examples for Flask to FastAPI and Python 2 to 3. Good for your rule-based first pass.

Five lessons that repeat everywhere:

1. Tests are the judge.
2. Use rules and codemods for predictable changes, and the LLM only for what is left.
3. Work in small chunks with small context. Never send a whole repo.
4. A human reviews what the system could not prove.
5. The orchestrator is ordinary code, not an LLM deciding who goes next.

4. How yours is different (honestly)

What we are NOT claiming: a new idea. This space is crowded and has big-company versions. If someone says it already exists, agree and move on to the points below.

What we claim instead:

1. **One pipeline, two very different recipes.** Python 2 to 3 is a same-language upgrade. Express to FastAPI crosses language and framework. Making both run on the same orchestrator proves the design generalizes, which most repos do not show.
2. **Evaluation is a first-class feature.** A held-out benchmark, repeated runs, and experiments that compare LLM-only against rules plus LLM, no retries against three retries, and one agent against separated agents. The last comparison answers the question every interviewer asks: why multi-agent.
3. **It is a real backend service,** not a script: job and chunk state machine in Postgres, workers claiming chunks safely in parallel, idempotent retries, sandboxed execution, tracing, an API.
4. **Honest failure reporting.** Chunks that could not be proven go to a human review queue with a reason, instead of being quietly marked done.
5. **Test isolation.** The test-writer never sees the migrated code, and the migrator never sees the held-out tests.

5. What this project shows that you understand

Skill	Where it shows up	Interview line
Backend design	Postgres job and chunk state machine, API, workers	I modeled migration as a state machine so any chunk can be retried or resumed without side effects
Concurrency	Workers claim chunks with row locks	I tested with 4 workers and verified no chunk was processed twice
Reliability	Retry limits, rollback, idempotency keys, failure injection	I killed workers mid-chunk and the job still finished
Security thinking	Docker sandbox with no network, memory and time limits	Generated code is untrusted, so it runs in an isolated container
Multi-agent design	Separate roles, tools, and permissions, deterministic orchestrator	Each agent only has the tools its job needs, and the orchestrator is plain code
LLM engineering	Structured outputs, response cache, cost caps, prompt versioning	I cache by prompt hash so experiments are reproducible and cheap
Testing mindset	Characterization tests, held-out benchmark	I did not trust the LLM, I trusted recorded behavior
Measurement	Experiment table with numbers	Rules plus LLM raised first-try pass rate from X to Y and cut cost by Z
Language internals	Py2 vs Py3 semantics, Express vs FastAPI behavior	I can explain why `/` and `str` and `bytes` break migrations

6. Architecture overview

```
           POST /jobs (repo + recipe)
                    |
              [ FastAPI API ]  <---- GET status, chunks, trace, report
                    |
              [ Postgres ]  jobs, chunks, attempts, agent_calls
                    ^
                    | claim chunks (FOR UPDATE SKIP LOCKED)
           [ Worker processes ]
                    |
         [ Orchestrator (plain code) ]
analyze -> plan -> record tests -> rules pass -> migrate -> verify
                    |                              ^           |
                    |                              +--retry----+
                    v
[ Agents + tools ]            [ Docker sandbox ]
Analyzer   Planner            runs old code, tests, new code
Test-writer Migrator          no network, memory and time caps
Verifier (tools + triage)
```

The agents (each has different tools and permissions):

Agent	Tools	Must NOT
Analyzer	file reader, parser, dependency graph	edit code
Planner	dependency graph, size estimates	read full file bodies
Test-writer	old code, sandbox running the OLD app	see new code
Migrator	read chunk, read signatures of dependencies, write one chunk	run tests or judge itself
Verifier	sandbox, test runner, diff tool	change code

Be honest about this in interviews: the Analyzer and Planner are mostly deterministic code, and the Verifier is mostly a tool. The LLM is used where judgment is needed: proposing test inputs, rewriting hard code, and explaining failures. Calling all five agents is fine if each has its own role, tools, and permissions, but say plainly where there is no LLM inside.

Chunk state machine:

`PENDING -> TESTS_READY -> RULES_APPLIED -> MIGRATING -> VERIFYING -> PASSED`

From `VERIFYING`, a failure goes to `RETRY` (back to `MIGRATING`, with the trimmed error) until the attempt limit, then to `NEEDS_REVIEW` with a reason. `FAILED` is for infrastructure errors such as a sandbox crash.

Why the orchestrator is plain code: an LLM deciding what to do next can loop, skip steps, or burn money. Code can enforce retry limits, budgets, ordering, and rollback every time, and it is easier to test and explain.

7. Key technical decisions and the alternatives

Decision	Our pick	Other options	Why our pick
Main language	Python 3.11+	Go, TypeScript	Best LLM and tooling ecosystem, and FastAPI is already on your resume
API framework	FastAPI	Flask, Django REST	Async, automatic docs, typed models
Database	Postgres	SQLite, MongoDB	Row locking, transactions, and constraints protect the state machine. SQLite is fine for a first spike
Job queue	Postgres as the queue using `FOR UPDATE SKIP LOCKED`, plus worker processes	Celery with Redis, RQ, Arq, Temporal, Kafka	Fewer moving parts and you learn real locking. Celery or RQ are valid. Temporal and Kafka are overkill here
Sandbox	Docker via the Docker SDK	Plain subprocess, gVisor or Firecracker, hosted sandbox services	Real isolation with limits. Subprocess is unsafe for untrusted code. The others are heavier or paid

Decision	Our pick	Other options	Why our pick
LLM access	Thin provider interface of your own, optionally LiteLLM underneath	LangChain, LangGraph, PydanticAI	You can explain every line. Frameworks hide the loop you need to talk about. LangGraph is fine inside one agent, but keep the orchestrator your own
Code parsing	Py2: analyzer script run inside a Python 2.7 container. JS: tree-sitter or acorn	Regex, an LLM to read structure	Regex breaks on real code and LLM parsing is slow and unreliable
Graph work	networkx	Hand-written graph code	Topological sort and cycle detection are built in
Tests	pytest, httpx, golden JSON files	unittest, Postman or Newman	Easy to script and compare
Tracing	A table plus an API endpoint first, then OpenTelemetry or Langfuse if time allows	Only logs	A table is queryable and shows up in the demo
Models	Cheaper model by default, stronger model on the last retry	One model for everything	Cost control with a measurable effect. Use a free local model for development and a cloud model for the final benchmark
UI	None at first, then a small read-only dashboard	Full React app	Backend depth is the point, do not burn time on UI

Part 2: The Build Plan

Timeline and where to start

This assumes part-time work. Stretch or shrink it to your own schedule, but keep the order.

Weeks	Phases	You have at the end
1	0 to 3	Repo, benchmark apps, DB state machine, safe sandbox
2	4 to 7	Analyzer, planner, recorded tests, rules pass for Python 2 to 3
3	8 to 10	Working command-line pipeline for Python 2 to 3 (first resume-worthy milestone, label it in progress)
4	11 to 12	API, workers, tracing
5	13 and start of 14	Evaluation harness and first experiment numbers
6	14 to 16	Express to FastAPI recipe, hardening, README, demo

Start here: your first three days

- **Day 1:** create the repo skeleton and a `docker-compose.yml` with Postgres. Write the one-page spec (phase 0). Write the first Python 2 benchmark module (text utilities) with about 10 tests, and run them inside a `python:2.7` container to confirm they pass on the old code.
- **Day 2:** write the sandbox runner with its four safety tests (infinite loop, memory bomb, no network, no writes outside the work folder). Run the Python 2 tests through it.
- **Day 3:** spike the analyzer: a small script run inside the Python 2 container that reads one file and prints its imports and function names as JSON. This is the riskiest early unknown, so find out now whether it works.

By the end of day 3 you have old code running safely in a sandbox and the first piece of the dependency graph. That is a real foundation, and nothing so far needs an LLM.

Phase 0: Scope, metrics, repo skeleton

What we build

- A one-page spec: the two recipes, the success metrics, and the non-goals.
- Metrics: first-try pass rate, pass rate after retries, human-review rate, regressions (tests that passed on old code and fail on new), LLM calls per chunk, tokens and cost, wall time.
- Non-goals: no UI beyond a read-only dashboard, no auto-merge, no third recipe until everything else is done.
- Repo layout:

```
lift/
  lift/
    api/            FastAPI app
    core/           orchestrator, state machine, config
    agents/         analyzer, planner, test_writer, migrator, verifier
    llm/            provider interface, cache, cost tracking
    sandbox/        docker runner
    recipes/
      py2_to_py3/
      express_to_fastapi/
    db/             models, migrations
  benchmarks/       legacy apps and held-out tests
  experiments/      eval runner and results
  tests/
  docker/
  docker-compose.yml
  pyproject.toml
```

Why we do it: deciding the metrics before you see results stops you from moving the goal after the fact. Non-goals protect your time. A clean layout makes the second recipe a plug-in instead of a rewrite.

Other options: pip with venv, Poetry, or uv for packaging (any is fine, pick one). You can keep benchmarks in a separate repo to keep held-out tests private, but a public benchmark folder makes your results more credible.

Done when: `pytest` runs one dummy test, `docker compose up` starts Postgres, linting is set up (ruff, pre-commit), and the spec is in the README.

Phase 1: The benchmark corpus (build the old apps first)

What we build

Python 2 benchmark app: 10 to 15 modules, for example text utilities, money and statistics calculations, CSV and file handling, a small inventory domain with classes, a report generator, and a tiny command-line entry point. Plant these known gotchas on purpose and give each an id and a difficulty (easy, medium, hard):

- `print` statement, `raw_input`, `xrange`, `except E, e`
- `/` integer division, `long`, old octal literals
- `str` vs `unicode` vs `bytes`, especially in file reading and hashing
- `dict.keys()`, `.items()`, `.has_key()`, `.iteritems()`
- `map`, `filter`, `zip` returning iterators in Python 3
- sorting with `cmp`, comparing mixed types (works in Python 2, error in Python 3)
- `__nonzero__`, `urllib` and `urllib2`, `StringIO`, implicit relative imports

Express benchmark app: 12 to 15 routes, SQLite or in-memory data, an auth middleware, request validation, custom error handlers, pagination and query params, and one callback-style async function. Gotchas to plant: error body shape, 422 vs 400 validation behavior, trailing slash handling, type coercion of query params, status code defaults, middleware order.

Two test suites per app: a **dev suite** the pipeline may see, and a **held-out suite** the pipeline never sees and only the evaluation harness runs. Confirm both pass on the old code, inside containers.

Why we do it first: you cannot build or judge an agent without knowing what correct means. Writing the legacy code yourself also teaches you exactly what breaks, which is what you will explain in interviews. The held-out suite stops you from fooling yourself, because a system can overfit to the tests it can see.

Other options:

- Real open-source legacy repos: more credible but much harder to get running and tested. Good as a stretch goal with two or three small repos (check licenses).
- LLM-generated legacy apps: fast but unrealistic, and the same model may then migrate its own style of code too easily.
- Our choice is a hybrid: your own seeded apps for control, plus a few real repos later for credibility.

Done when: both suites pass on the old code in containers, and `GOTCHAS.md` lists each seeded gotcha with an id and difficulty so results can later be reported per difficulty.

Phase 2: Data model and state machine

What we build (Postgres, SQLAlchemy plus Alembic migrations):

- `jobs`: id, recipe, repo reference, status, token budget, timestamps
- `chunks`: id, job id, file or symbols, dependency level, status, attempt count, review reason
- `attempts`: id, chunk id, attempt number, input hash, result, trimmed error
- `agent_calls`: id, job id, chunk id, agent, model, prompt hash, tokens in and out, latency, cost, tool name, ok flag
- `artifacts`: id, job id, kind (diff, report, golden tests), storage path
- One function `transition(chunk_id, from_status, to_status)` that runs `UPDATE ... WHERE status = from_status` and returns whether it changed a row. Invalid or repeated transitions change nothing.

Why we do it: the database is the backbone of reliability. If a worker dies, the chunk is still in a known state and can be resumed. Retries are tied to attempt numbers and input hashes, so running the same step twice cannot corrupt anything. Tracing is just queries over `agent_calls`.

Other options:

- SQLite for the first spike (simple, but weaker for concurrent workers).
- Redis for state (fast, but weaker guarantees and harder to query history).
- Plain in-memory state (easiest, but a crash loses everything).
- Event sourcing (powerful, overkill for this scope).
- ORM vs plain SQL: an ORM is faster to write, while plain SQL makes the locking logic clearer to explain. Mixing SQLAlchemy Core for the hot queries is a good middle path.

Done when: tests show an invalid transition is rejected, and two concurrent attempts to move the same chunk result in exactly one success.

Phase 3: Sandbox runner

What we build: `run(image, mounts, cmd, limits) -> Result` using the Docker SDK for Python.

- Limits: no network, memory cap, CPU cap, process count cap, read-only root filesystem with a writable work folder, non-root user, dropped capabilities, and a wall-clock timeout that kills the container.
- Images: `python:2.7` for old Python code, `python:3.11` for new code, `node:20` for the Express app. Check that each image can still be pulled in your environment.
- Result: exit code, truncated stdout and stderr, duration, and a `timed_out` flag.

Why we do it early: both the recorder and the verifier depend on it, and generated code is untrusted. Running it directly on your laptop is a real security risk, and it is also the first thing an interviewer will ask about.

Other options:

- Plain `subprocess` with a timeout: simplest, no isolation. Fine for a throwaway spike, not for the project.
- Docker (our pick): real isolation with simple limits.
- gVisor or Firecracker: stronger isolation but heavy setup.
- Hosted sandbox services: easy but paid, and your code leaves your machine.
- If workers run inside containers themselves, they need Docker access. Mounting the Docker socket gives broad host access, so note that trade-off in your README.

Done when: four safety tests pass: an infinite loop is killed, a memory bomb is stopped, a network call fails, and a write outside the work folder fails.

Phase 4: Analyzer (deterministic, no LLM)

What we build: for each recipe, a JSON graph of the repo: files, imports, top-level functions and classes, call edges where cheap to find, and for Express the routes, middleware, and requires. Then use networkx to find cycles and compute topological levels (level 0 files import nothing from the repo).

The Python 2 trap: Python 3's `ast` module cannot parse Python 2 syntax, so a `print` statement is a syntax error. Options: run the analyzer script inside the `python:2.7` container and use its own `ast`, then print JSON; or use the older `lib2to3` grammar, which can read Python 2 (it is deprecated and removed in newer Python versions, so check your environment). Spike this on day 3.

Express analysis: a small Node script using acorn or `@babel/parser` to find `app.get/post/...` calls and `require` statements, or tree-sitter from Python.

Why we do it: the planner needs dependency order. If you change a function signature, its callers break, so you migrate the leaves first. It also keeps each agent's context small.

Other options:

- Regex: quick, but it breaks on real code (multi-line imports, strings that look like code).
- An LLM reading the repo: slow, costly, and can miss or invent edges.
- Existing tools (pydeps, madge for JavaScript): fine as a baseline to check your own graph.
- Cycles: collapse a cycle into one chunk so it is migrated together.

Done when: the graph for the benchmark app matches a hand-checked expected graph in a test.

Phase 5: Planner and chunker

What we build: turn the graph into chunks and a context pack per chunk.

- Chunk = one file by default. Split large files by top-level function. Respect a token budget per chunk.
- Context pack = the chunk's source, the signatures (not bodies) of what it imports, its recorded test cases, and a short list of recipe hints (the known gotchas for that recipe).
- Order = by dependency level. Chunks on the same level do not depend on each other and can run in parallel.
- Optional: an LLM step that adds risk notes for chunks with tricky constructs, so retries are spent where they matter.

Why we do it: LLMs do badly on whole repositories and well on small, well-specified chunks. Chunking and context packing is the biggest quality lever in the whole project.

Other options: file-level chunks (simple, our start), function-level (finer but more calls), package-level (fewer calls, bigger context, worse accuracy), LLM-decided chunks (flexible but unpredictable).

Honest note: the planner can be 100 percent deterministic code. Adding an LLM only for risk notes is fine, but do not put an LLM where plain code already does the job.

Done when: the benchmark app produces chunks in a valid order, each context pack is under the token budget, and parallel-safe levels are marked.

Phase 6: Recording tests on the old code (characterization tests)

What we build:

Test-writer agent proposes inputs. For Python 2, an LLM reads the old code and proposes calls with edge-case arguments (empty, negative, unicode, very large, mixed types) as a structured list. For Express, an LLM reads the route handlers and proposes HTTP requests (valid, missing field, bad id, wrong types, no auth).

Recorder runs them on the OLD code. The recorder executes the proposed calls in the sandbox against the old code and stores the real outputs (return values, printed output, exceptions) as golden files. For Express it starts the old app in a container, sends the requests, and stores status code, selected headers, and JSON body.

Comparator rules. Ignore the `Date` header and key order, normalize generated ids and timestamps, and compare error shapes strictly.

Why we do it: this is the heart of the project. The LLM proposes inputs, and real execution of the old code decides the expected outputs, so you never trust the LLM to know the right answer. Recording on old code only also avoids tests that quietly match the new code.

Other options:

- Only hand-written tests: very reliable, but not a product feature. You already have these as the dev and held-out suites.
- LLM-written expected outputs: tempting, and risky because the LLM can be confidently wrong.
- Property-based testing with Hypothesis: a good extra for pure functions.
- Recording real production traffic: the best approach in real companies, and out of scope for a student project (mention it in your README as future work).

Warning on non-determinism: time, randomness, file system, and network make outputs unstable. Freeze or mock them in the recorder, and mark chunks you cannot test as `NEEDS_REVIEW` instead of pretending they passed.

Done when: the recorded tests pass on the old code three runs in a row, and every function in the benchmark is covered at least once.

Phase 7: Rules pass (cheap and deterministic first)

What we build:

- *Python 2 to 3:* run a codemod tool for the mechanical changes. Options are the `2to3` tool (from `lib2to3`, check availability in your Python version), `python-modernize` or `futurize`, or your own small fixes for the six or seven most common gotchas. Run it in a container that has the tool.
- *Express to FastAPI:* generate FastAPI route stubs from the route table: `:id` becomes `{id}`, methods and status codes copied, Pydantic models drafted from observed request bodies. Leave handler bodies for the migrator.
- The rules pass output goes through the same verifier as everything else.

Why we do it: public write-ups on large migrations repeat the same finding: deterministic changes are cheaper, faster, and repeatable, and the LLM should handle what is left. This also becomes experiment E1 in phase 13.

Other options: skip the rules pass (simpler and more expensive; keep a flag so you can switch it off for the experiment), use LibCST or jscodeshift for your own codemods, or write everything custom.

Done when: the rules pass alone fixes the easy gotchas in the benchmark, and a report lists which recorded tests still fail afterwards.

Phase 8: LLM layer

What we build: one interface, `complete(messages, tools=None, schema=None, model=...)`, that hides the provider.

- Retry with exponential backoff on rate limits and timeouts.
- Structured output validated with Pydantic. If the JSON is invalid, re-ask once with the error, then fail the call.
- A disk cache keyed by a hash of model, prompt, and parameters, so reruns are free and reproducible.
- Token and cost counting written to `agent_calls`, plus a per-job token budget that raises `BudgetExceeded`.
- Prompts stored as files in the repo (`prompts/migrator_v1.md`) with the version recorded on every call.
- A fake provider that returns canned outputs, so tests and CI run without API keys.

Why we do it: provider errors, cost, and reproducibility are the real engineering work in LLM apps. The cache makes experiments cheap and repeatable, and the fake provider lets you unit test the orchestrator.

Other options:

- LiteLLM underneath your interface: one API across providers, saves time.
- LangChain: large, and hides details you will be asked about.
- One SDK directly: simple, but locks you to one provider.
- Local models through Ollama for development: free but weaker. Use a stronger cloud model for the final benchmark numbers.
- Use a low temperature for stability, but remember outputs are still not perfectly deterministic. That is why experiments run several times.

Done when: tests with the fake provider cover retry, invalid JSON, cache hit, and budget stop.

Phase 9: Migrator and Verifier agents

Migrator: what we build

- Input: the context pack, the rules pass output, and on retries the trimmed failing test report and its previous attempt.
- Tools: `read_signature(symbol)`, `read_file(path)` (only files in the chunk's dependency list), and `write_chunk(content)` (only the chunk's own file).
- Not allowed: running tests, editing other files, network.
- Prompt: role, rules (preserve behavior, do not refactor, do not change public signatures, keep comments), the recipe hints, two or three short gotcha examples, and a strict output schema. The tool loop has a step cap.

Verifier: what we build (mostly deterministic)

1. Compile or syntax check of the new file.
2. Run the recorded tests for this chunk against the new code in the sandbox.
3. Re-run the tests of chunks that already passed, to catch regressions.
4. Produce a trimmed failure report: the first few failing cases, expected vs actual, and a short stack trace.
5. Optional LLM triage: classify the failure (syntax, semantic, environment) and write a one-paragraph hint for the migrator.

Why we do it: the agent that writes code must not judge its own code. Restricting tools limits damage and makes failures easy to debug. Be honest in interviews that the verifier is a tool with optional LLM triage, not an LLM deciding pass or fail.

Other options:

- One agent with all tools: simpler, and exactly what experiment E3 compares against.
- More agents (style reviewer, security checker): good stretch features.
- LLM-as-judge for semantic equivalence: weaker than tests. Use it only as an extra signal, never as the pass criterion.
- Diff-style edits instead of rewriting the whole file: saves tokens on large files but is easier to get wrong. Whole-file rewrite is fine for small chunks.

Done when: one benchmark chunk migrates end to end, and the verifier correctly fails a deliberately broken output.

Phase 10: Orchestrator (the control loop)

What we build: plain code that drives each chunk through the states. Pseudo-code:

```
for level in chunks_by_level:
    for chunk in level:                      # parallel later, in phase 11
        tests = record_tests(chunk)          # phase 6, cached
        snapshot(chunk)                      # git commit or file copy
        code = rules_pass(chunk)
        for attempt in 1..MAX_ATTEMPTS:
            if budget_exceeded(job):
                mark(chunk, NEEDS_REVIEW, 'budget'); rollback(chunk); break
            code = migrator(chunk, code, last_report)
            result = verifier(chunk, code)
            log(attempt, result)
            if result.passed and not result.regressions:
                commit(chunk, code); mark(chunk, PASSED); break
            if same_failure_as_last(result):
                mark(chunk, NEEDS_REVIEW, 'stuck'); rollback(chunk); break
            last_report = result.trimmed_report
        else:
            mark(chunk, NEEDS_REVIEW, 'max_attempts'); rollback(chunk)
```

Rules baked in:

- Max 3 attempts. The last attempt uses the stronger model.
- Budget per chunk and per job.
- Loop detection: the same failure twice means stop.
- Rollback restores the snapshot. A chunk that breaks earlier passing tests cannot pass.

- Every state change goes through `transition()`.
- Idempotency key per attempt: chunk id, attempt number, input hash.
- Review reasons are a fixed list: `tests_unrecordable`, `max_attempts`, `stuck`, `budget`, `sandbox_error`, `regression`.

Why we do it: limits, ordering, and rollback must be guaranteed, so they live in code, not in a prompt. It is also easy to unit test with the fake LLM.

Other options:

- LangGraph state graph: you already know it, it draws nice diagrams, but it hides some of the control flow you want to explain.
- Temporal: durable workflows out of the box and excellent, but heavy for this project.
- Event-driven agents passing messages on a queue: scales better, is much harder to reason about.
- For now the loop runs in one process. Phase 11 moves it into workers.

Done when: a fake LLM that fails twice then succeeds is retried correctly, a fake that never succeeds ends in `NEEDS_REVIEW` with a rollback, and the command `lift run benchmarks/py2_app --recipe py2-to-py3` produces a migrated repo plus a report.

This is Milestone 1. You now have a working Python 2 to 3 pipeline. If you run short of time, you can stop here, add evaluation numbers, and still have a complete project.

Phase 11: Service layer (API, workers, queue)

What we build

API endpoints (FastAPI):

- `POST /jobs`: zip or git URL, recipe, token budget. Returns a job id.
- `GET /jobs/{id}`: status and counts.
- `GET /jobs/{id}/chunks`: per-chunk status, attempts, and review reason.
- `GET /jobs/{id}/trace`: timeline of agent calls.
- `GET /jobs/{id}/artifact`: migrated repo zip plus report.
- `POST /jobs/{id}/chunks/{cid}/retry` and `/approve`: human review actions.
- `GET /jobs/{id}/events`: server-sent events for live progress.

Workers: separate processes that claim ready chunks with a row lock:

```
SELECT id FROM chunks
WHERE status = 'PENDING' AND dependencies_done
ORDER BY level
FOR UPDATE SKIP LOCKED
LIMIT 1;
```

- A claimed chunk gets a lease with a heartbeat. A reaper puts chunks whose lease expired back to `PENDING`, which is safe because attempts are idempotent.
- A chunk is claimable only when everything it depends on has passed.
- Input safety: upload size limit, reject zip paths containing `..` (zip slip), allow only expected file types, an API key header, and a simple rate limit.

Why we do it: this is where the backend depth shows: concurrency, leases, idempotency, and input safety. It also turns the project from a script into a service others can call.

Other options:

- Celery with Redis: mature, with built-in task retries, but more infrastructure.
- RQ or Arq: lighter queues.
- Postgres as the queue (our pick): fewer parts and you learn locking.
- Temporal: best durability, heavy.
- Kafka: right for very high volume event streams. Overkill here, but you can say how you would use it to scale.
- Auth: an API key is enough. OAuth is not worth the time here.

Done when: a run with 4 workers on the benchmark shows every chunk processed exactly once (check the attempts table), and killing a worker mid-chunk still lets the job finish.

Phase 12: Observability

What we build

- Every LLM call and tool call is already a row in `agent_calls`. Add structured JSON logs carrying `job_id` and `chunk_id`.
- The `/trace` endpoint returns a timeline per chunk.
- A small read-only dashboard (Streamlit or one HTML page): chunk grid colored by status, cost per chunk, time per stage.
- Optional: Prometheus counters, OpenTelemetry spans, or Langfuse.

Why we do it: debugging agents without traces is guessing. It also makes the demo convincing, because you can show exactly what happened in a run.

Other options: logs only (cheap, hard to query), Langfuse or LangSmith (good LLM-specific views, external dependency), OpenTelemetry with Jaeger (standard backend skill), a DB table (our first step).

Done when: for any job you can answer which chunk cost the most, which stage is slowest, and which agent fails most.

Phase 13: Evaluation harness

What we build: `experiments/run.py --config e1.yaml --runs 3` creates jobs on the benchmark, waits, runs the **held-out** suite on the output, and writes a CSV and a markdown table.

Experiments:

- **E0 baselines:** rules only, and a one-shot LLM rewrite of each file with no tests or retries.
- **E1:** LLM only vs rules plus LLM.
- **E2:** 0 vs 1 vs 3 retries.
- **E3:** one agent with all tools vs separated roles. This is the experiment that honestly justifies multi-agent. Report the result even if the difference is small.
- **E4:** cheap model vs strong model vs cheap-then-strong escalation, measured as cost per passed chunk.

Metrics: first-try pass %, pass after retries %, human-review %, regressions, all split by gotcha difficulty, plus LLM calls per chunk, tokens and cost, and time. Run each configuration 3 to 5 times and report the mean and range. Score the recorded suite and the held-out suite separately. If recorded tests pass but held-out tests fail, your recorded tests are too weak, and that is worth reporting.

Why we do it: numbers make the project credible, and honest reports with negative results are rare and impressive.

Other options: only a demo video (weak), manual spot checks (do these too: read 10 outputs line by line), formal statistical tests (overkill, mean and range is enough), a bigger benchmark of real repos (stretch). Use the response cache and a small benchmark first to keep costs low.

Done when: the README has a results table with at least E1 and E2 for Python 2 to 3, with repeated runs.

Phase 14: Second recipe, Express to FastAPI

What we build: implement the same recipe interface:

```
class Recipe:
    name, source_image, target_image
    analyze(repo) -> Graph
    rules_pass(chunk) -> code
    propose_tests(chunk) -> list[TestCase]     # LLM step
    record(chunk) -> Golden                    # runs against OLD app
    compare(golden, actual) -> Result          # normalization rules
    hints: list[str]                           # gotchas for the prompt
```

Reused unchanged: orchestrator, database, workers, API, sandbox, LLM layer, tracing, evaluation harness.

New: Node analyzer, route stub generator, HTTP recorder and comparator, FastAPI-specific prompt hints. The tests run inside the same container as the app, so the sandbox can keep networking off except loopback.

Express chunking: shared modules (DB, utils, middleware) first, then route files.

Gotchas for the hints: error body shape (`detail`), 422 validation errors, trailing slash redirects, query parameter coercion, `async def` vs `def`, middleware becoming dependencies, status code defaults, CORS.

Why we do it: a second recipe proves the design generalizes, which is a stronger story than one recipe done deeply. If the recipe interface needs changes while you build it, write those down in the README, because that is genuine design learning.

Other options for the second recipe: callback JavaScript to async/await (smaller), Flask to FastAPI (closer, and codemod examples exist), Java 8 to 17 (needs a JVM and a bigger benchmark), jQuery to React (hard to test behavior). We pick Express to FastAPI because you already work with both stacks and HTTP tests do not care about the server language.

Done when: the results table has E1 and E2 for both recipes, and the README shows which code was shared.

Phase 15: Hardening, CI, security

What we build

- Failure-injection tests: kill a worker mid-chunk, LLM returns malformed JSON or nothing, sandbox timeout, database connection drop, the same job submitted twice, a huge upload, a zip containing `../`.
- GitHub Actions: lint, unit tests with the fake LLM, and one tiny end-to-end run on cached responses.
- Security checklist: sandbox flags, secrets in environment variables only, no API keys in logs, upload limits, pinned dependencies, and a note on the Docker socket trade-off.
- Docs: architecture diagram, a sequence diagram for one chunk, and short decision records (why a Postgres queue, why your own orchestrator, why Docker sandboxing). These decision records are excellent interview material.

Why we do it: reliability is what you claim, so prove it. Decision records make your reasoning visible.

Other options: skip CI (faster, but looks unfinished), integration tests with a real LLM (flaky and costly, run them manually), load testing the API with k6 or Locust (optional, gives concurrency numbers).

Done when: CI is green and the failure-injection suite passes.

Phase 16: Packaging, demo, resume, interviews

README order: one-paragraph pitch, demo GIF, results table, architecture diagram, how to run in three commands, design decisions, honest limitations, future work.

Three-minute demo: submit a job, watch the chunk grid update, open one failed chunk and its trace, show it in the review queue, then show the final report.

Resume bullet templates (use only numbers you actually measured):

- Built a multi-agent legacy migration service (FastAPI, Postgres, Docker) that converts Python 2 to 3 and Express to FastAPI, verifying behavior with tests recorded on the original code.
- Designed a Postgres-backed job queue with row-level locking and lease expiry, running [N] chunks across [W] workers with zero duplicate processing.
- Evaluated on [X] held-out tests: rules plus LLM reached [A]% first-try pass vs [B]% for LLM-only, at [C]% lower cost.

Questions to prepare for, with the short answer idea:

1. *Why multi-agent?* Separate roles, tools, and permissions, and experiment E3 shows the effect.
2. *How do you know the migration is correct?* Behavior recorded on the old code, plus held-out tests, plus human review for the rest.
3. *What if the old code had a bug?* The system preserves behavior and flags suspicious lines for review.
4. *How do you stop infinite loops and cost blowups?* Attempt cap, stuck detection, token budgets.
5. *What if a worker crashes?* Leases expire and the chunk is retried idempotently.
6. *How do two workers not take the same chunk?* `FOR UPDATE SKIP LOCKED` plus guarded transitions.
7. *Why Postgres, not Kafka or Celery?* Fewer parts and real locking. Here is when I would switch.
8. *How is the sandbox secured?* No network, resource limits, non-root, read-only filesystem, timeouts.
9. *What failed?* Have two real examples, such as a gotcha the system never solved.
10. *How would this scale to a million lines?* Function-level chunks, many workers, a real queue, and caching of recorded tests.

If time runs short: the cut list

Cut in this order: (1) the dashboard, (2) the second recipe, replaced by a smaller one or dropped, (3) function-level chunking, (4) LLM triage and risk notes, (5) OpenTelemetry. **Never cut:** the sandbox, recorded tests, the held-out evaluation, the state machine, and the retry limit with rollback. The minimum credible version is Python 2 to 3 with Postgres state, the sandbox, recorded tests, and E1 and E2 numbers.

Main risks and how to handle them

Risk	What to do
LLM output changes between runs	Cache by prompt hash, low temperature, repeat experiments 3 to 5 times

Risk	What to do
Scope creep	Vertical slice first, non-goals in the spec, the cut list above
Python 2 tooling friction	Spike the analyzer on day 3, use containers, keep a regex fallback only for the spike
Flaky tests from time, randomness, files	Freeze or mock in the recorder, mark untestable chunks for review
Cost	Response cache, per-job budget, small benchmark first, local model for development
Sandbox escape or abuse	Strict container flags, no network, document the Docker socket trade-off
Overclaiming on the resume	Claim only what passes held-out tests, say benchmark I built, report failures

Final honesty checklist

- Describe results as measured on a benchmark you built, not as production results.
- Be able to explain every file and every decision, using the alternatives above.
- Mark the project as in progress on your resume until Milestone 1 works and you have real numbers.
- Read the repos listed in section 3 for ideas on structure and reporting, but write your own code.